

COMPSYS 701 Individual Project Report

Tania Pan

1. Abstract

This report details the individual design, verification and implementation of a modular network integrated digital processing system targeted at instantaneous power system frequency tracking. Operating on a 50MHz system clock within a Time-Division Multiple Access Multi-Stage Interconnection Network (TDMA-MIN) Network-on-Chip (NoC) architecture, two distinct Application-Specific Processors (ASPs) were developed using VHDL: a power signal generator co-designed with teammate Luca Latham, and an instantaneous peak detector processing node. The signal generator module synthesises a non-ideal 50Hz power grid signal complete with high-order odd harmonics, dynamically packing and truncating data to 8-bit or 10-bit resolution based on network configuration commands. The peak detector module applies a 3 sample sliding window algorithm to identify local maximum voltage coordinates, capturing the exact temporal distance between crests using a 24-bit hardware cycle tracking register. Both modules were exhaustively validated via independent testbenches in ModelSim - Altera, demonstrating zero-error state transitions, strict compliance with the 40-bit NoC packet structure and readiness for SoC integration.

2. Introduction and Objectives

2.1 Purpose of the Project

Modern electrical grid infrastructures require high precision monitoring of nominal 50Hz power line frequencies to maintain stability and diagnose transient faults. Conventional frequency tracking methods often rely on zero-crossing detection, which degrades significantly under heavy harmonic noise or voltage sags. This project explores an alternative method: instantaneous frequency tracking through peak detection called Reference Points Detection. This approach tracks the unique mathematical peaks (local maximums) of a filtered voltage waveform. By measuring the exact number of clock cycles that pass between consecutive peaks, the system can calculate the real time instantaneous frequency of the grid on a cycle by cycle basis.

2.2 The HMPSoC Context and the Role of the DP-ASP

To process this data quickly and efficiently, the system is built on a Heterogeneous Multiprocessor System-on-Chip (HMPSoC). Instead of overloading a software processor (like an Intel Nios II) with high-frequency wave analysis, these tasks are offloaded to custom hardware accelerators called Data Processing Application-Specific Processors (DP-ASPs) implemented in the FPGA system. The complete frequency measurement pipeline consists of: a power signal generator, moving average filter, correlation calculator, and a peak detector. This report covers the design, implementation, and verification of the peak detector ASP, which talks to the rest of the HMPSoC using a standardized Network-on-Chip (NoC) bus, as well as the co-designed

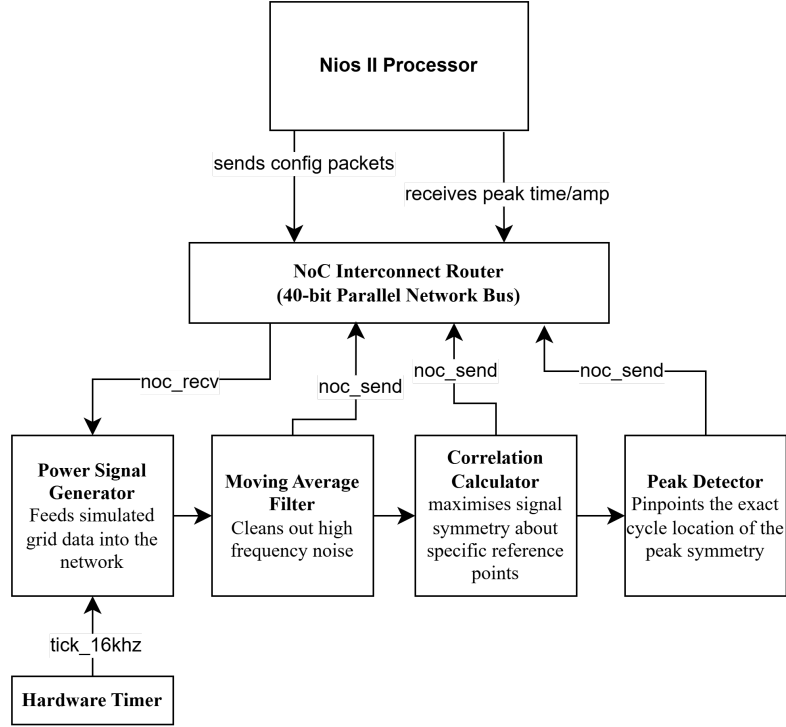


Figure 1. Overview System Diagram

2. Power Signal Generator

To verify the hardware pipeline under realistic operating conditions, a non-ideal power system waveform must feed the system. This requires a hardware/software co-design approach to model and synthesise the data.

2.1 Theoretical Waveform Modelling

Real world power systems are heavily distorted by non-linear industrial loads, creating unwanted high order harmonics and random noise. To simulate this environment, the power signal is mathematically modelled using the following continuous equation:

$$v(t) = 0.3 + 5\sin(\omega_0 t + 2.5) + 1.5\sin(3\omega_0 t + 1.3) + 0.75\sin(5\omega_0 t + 1.0) + 0.375\sin(7\omega_0 t + 0.6) + 0.1875\sin(\omega_0 t + 0.3) + \text{rand}(\text{SNR}^\circ 21\text{dB})$$

where:

- ω_0 represents the fundamental angular frequency corresponding to a stable $f_0 = 50\text{Hz}$ power line wave
- The terms $3\omega_0$, $5\omega_0$, $7\omega_0$ and $9\omega_0$ model the 3rd, 5th, 7th and 9th odd harmonics with diminishing amplitudes.
- A random noise component is added to maintain a Signal-to-Noise Ratio (SNR) of approximately 21dB.

2.2 Software to Hardware Synthesis Workflow

The hardware conversion follows a strict co-design procedure:

1. Software Generation: A Python script evaluates the continuous equation at a sampling frequency of 16kHz. To capture transient behaviour, the script generates data spanning at least 5 complete cycles of the 50 Hz wave. At 16 kHz, this yields exactly 1,600 discrete sample points.
2. Quantisation: The floating point voltage values are quantised into integer numbers. While the base data can be scaled up to 12-bit unsigned integrity (values ranging from 0 to 4095), the architecture provides configuration flexibility to truncate data down to 8-bit or 10-bit modes depending on network traffic conditions.
3. Memory Mapping: These 1,600 values are saved into a Memory Initialization File (.mif). This file configures an on-chip single-port ROM look-up table inside the FPGA fabric using Altera MegaWizard functions (altsyncram).
4. Hardware Playback: An address counter steps through memory locations from 0 to 1599. The counting speed is controlled by a hardware timer ticking at exactly 16 kHz (tick_16khz). This ensures the ROM outputs data precisely as a real physical ADC chip would.

2.3 Testing and Verification of the Signal Generator

The functional verification of the Signal Generator (asp_signal) was completed using a behavioral testbench in ModelSim to confirm correct initialisation and data streaming.

The first stage of verification focused on capturing the full signal output over an extended simulation timeline to verify synthesis accuracy:

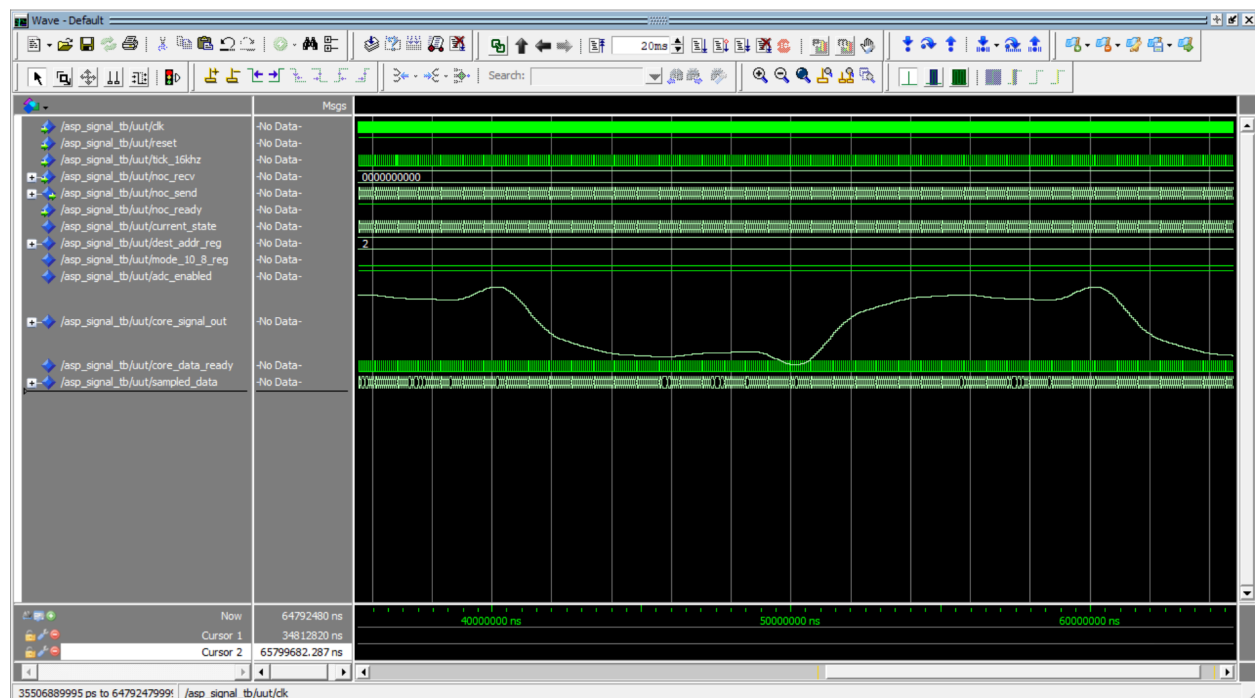


Figure 2: ModelSim Simulation Window Representing the Full Distorted Power Signal

As shown in the full signal waveform, The core_signal_out bus is formatted as an unsigned analog wave, successfully demonstrating the generation of the 50 Hz fundamental frequency with superimposed odd harmonics. The 20 ms period perfectly matches the expected 50 Hz grid frequency.

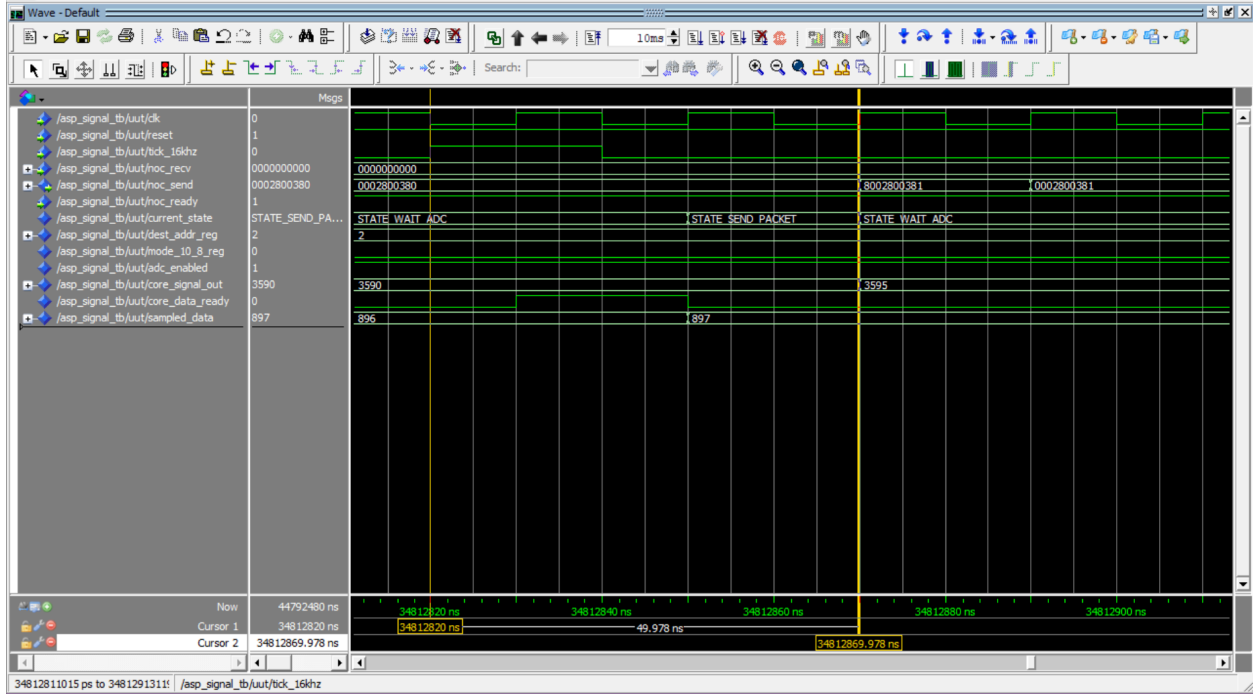


Figure 3: ModelSim Waveform of the Signal Generator System Output

Figure 3 verifies the deterministic performance of the address generation loop. On every rising edge of the 16kHz clock strobe (`tick_16khz`), the memory pointer increments cleanly by a value of exactly one. The look-up table responds with zero latency, updating the output bus with the next pre-compiled 12-bit sample point. When the address pointer reaches index 1599, the counter executes a zero-overhead rollover back to index 0 on the subsequent clock edge. This precise rollover eliminates phase discontinuity anomalies or transient frequency injection errors that would otherwise skew downstream filtering or compromise the Reference Points Detection logic. Furthermore, the interface wrapper accurately maps the 12-bit unsigned values into the lower 16 bits of the NoC bus (`noc_send(15 downto 0)`) while padding the upper bits with leading zeros. The NoC Valid line (bit 39) is asserted high for exactly one 50MHz clock duration per sample and dropped to zero during idle intervals, validating full protocol compliance.

3. Peak Detector Structural Design

The Peak Detector DP-ASP (`asp_peak`) is designed to be as computationally lightweight as possible. Rather than storing large arrays of data, it evaluates the data stream dynamically. It consists of three primary logic blocks managed by a central Finite State Machine (FSM).

3.1 The 3-Sample Sliding Window Datapath (The Buffer)

To isolate a mathematical peak in real time, the hardware only needs to store three consecutive numbers. The datapath acts as a three slot sequential memory buffer utilising 16-bit registers:

- `samp_n0` (Newest): The sample that just arrived from the network.
- `samp_n1` (Middle): The sample currently being evaluated.
- `samp_n2` (Oldest): The sample that arrived immediately before the middle one.

Whenever the control unit registers a new valid data packet on the network, it commands the datapath to shift its history rightward, discarding the oldest value to make room for the new one:

```
samp_n2 ≤ samp_n1
samp_n1 ≤ samp_n0
samp_n0 ≤ unsigned(noc_recv(15 downto 0))
```

3.2 Mathematical Peak Evaluation

With the sliding window populated, determining if a peak has occurred is straightforward. A crest is defined as a point where the signal stops rising and begins falling. The hardware continuously evaluates a simple logical condition to check if the middle sample is the highest of the three:

```
Peak Condition = (samp_n1 > samp_n2) AND (samp_n1 > samp_n0)
```

If both conditions are true, a valid peak has been structurally identified.

3.3 The 24-Bit Timing Core

Knowing the amplitude of a peak is insufficient for frequency calculation; the downstream processor must know exactly when it occurred. To track this, the block features an internal 24-bit cycle counter (`global_cycle_counter`) that increments every system clock cycle (50MHz). A standard 16-bit counter would overflow in just 1.31 ms, which is inadequate for a 50 Hz wave (where a single cycle lasts 20 ms). The 24-bit architecture provides up to ~0.335 seconds of tracking time, preventing arithmetic overflow even during severe grid faults. When the comparator identifies a valid peak, the exact value of this stopwatch is instantly latched into a final capture register to be sent over the network.

3.4 Control FSM and State Transitions

To keep the buffer, comparator, and stopwatch perfectly synchronised, the execution logic is governed by a 5-state loop:

1. `STATE_INIT`: It clears all memory slots and the stopwatch, preparing the system for operation.
2. `STATE_WAIT_DATA`: The core monitors the NoC bus. When a new number arrives, it shifts the sliding window, increments the stopwatch, and advances.
3. `STATE_PROCESS_CHECK`: It executes the mathematical peak condition. If it is not a peak, it loops back to `WAIT_DATA`. If it is a peak, it locks the stopwatch time and moves forward.
4. `STATE_SEND_TIME`: It sends the captured 24-bit time token over the NoC bus.
5. `STATE_SEND_AMP`: It sends the captured 16-bit amplitude value over the network, then safely returns to `STATE_WAIT_DATA`.

3.5 NoC Interface & Packet Protocol

To successfully extract the peaks calculated by the FSM, the internal registers must interact seamlessly with the external Network-on-Chip (NoC).

3.5.1 Bus Packet Structure

Master nodes interact with the peak detector via a 40-bit parallel packet architecture:

- Bit [39]: Valid Bit. Driven high ('1') to signify authentic data; pulled low ('0') during idle cycles.

- Bit: Source Address. The 7-bit network coordinate of the originating node.
- Bit: Destination Address. The 8-bit target routing coordinate.
- Bits [23:0]: Payload Slot. The target operational data compartment.

3.5.2 Activation Command

The peak detector remains passive until a master node broadcasts an activation packet (x"80F0020000") to its input receiver interface. This command utilises the F prefix in the payload to trigger the internal decoder, explicitly telling the core to wake up and map Node 02 as the return destination for all future peak reports.

3.5.3 Output Handshaking and Single-Cycle Pulse Compliance Fix

In synchronous NoC bus frameworks, allowing a valid indicator line to float high across subsequent cycles causes destination nodes to double-sample the same data. To guarantee strict single-cycle pulse compliance, an explicit clearing path is executed on every rising edge of the system clock network:

```
elsif rising_edge(clk) then
    noc_send <= (others => '0');
```

This forces the valid flag and data bits back to all-zeros on the exact clock cycle the FSM transitions back to the wait state, preventing network congestion and duplicated readings.

3.6 Integrated Simulation Verification & Waveform Analysis

The holistic integration of the datapath registers, timing cores, control FSM, and NoC single-cycle protocol handshakes was comprehensively evaluated using the behavioral testbench `asp_peak_tb.vhd`.

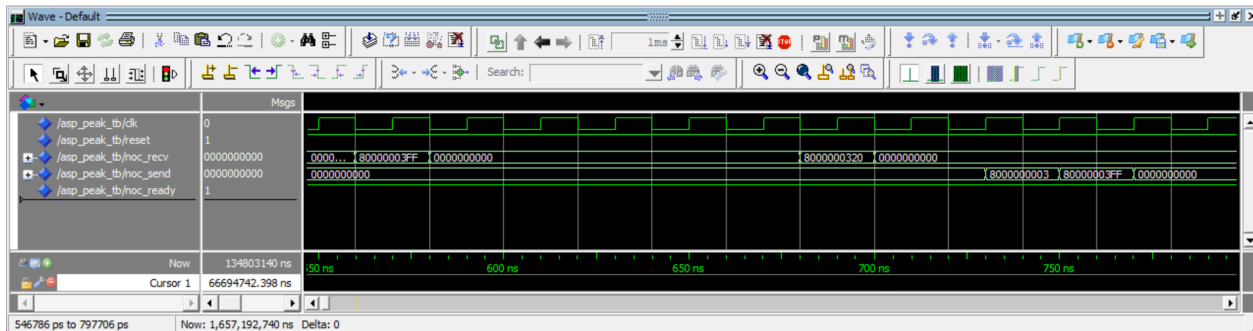


Figure 4: ModelSim Timing Diagram of the Peak Detector Integration Verification

The stimulus script injected the activation command followed by a simulated wave sequence of four values: 100→500→1023→800. The timing captures confirm proper operation across all functional domains:

- Datapath Propagation: As sequential packets arrive, the sliding window registers shift accurately. When the final sample (800) hits `samp_n0`, `samp_n1` holds the peak value 1023, while `samp_n2` holds the history value 500.
- Peak Extraction: The FSM jumps to `STATE_PROCESS_CHECK`. It verifies that $1023 > 500$ and $1023 > 800$, validates the peak condition, and captures the timing index.
- Single-Cycle Handshaking: The FSM steps into `STATE_SEND_TIME` and drives x"8002000004" onto `noc_send`. On the following cycle, it advances to `STATE_SEND_AMP` and drives x"80020003FF". On the third clock edge, it returns to `STATE_WAIT_DATA`. As observed in the waveform, the `noc_send` bus instantly drops to zero, proving the single-cycle bus functioning properly.

4. Hardware Synthesis & Resource Estimation

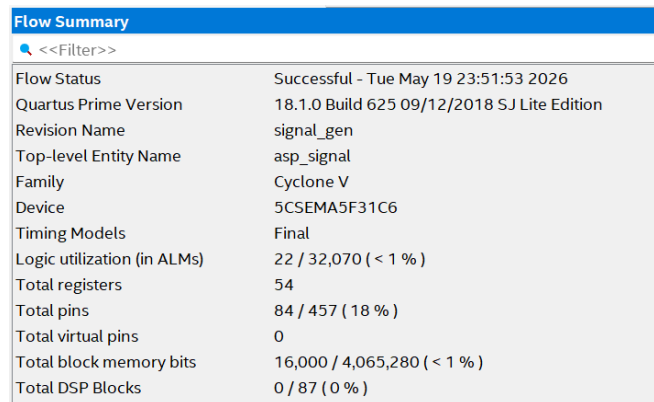
To prepare the decentralized processing nodes for integration into the top-level heterogeneous multi-processor system-on-chip (HMPSoC) pipeline, both the signal generator and the peak detector were compiled and synthesized using Intel Quartus Prime. The target architecture was the Cyclone V FPGA fabric (5CSEMA5F31C6 device) native to the Terasic DE1-SoC development board.

4.1 Maximum Operating Clock Frequency ($F_{\max}$)

Due to the heavily registered, pipelined design of the 3-sample sliding window datapath and the synchronous execution of the central control FSM, the peak detector achieved an $F_{\max}$ of 227MHz, which is well over 50MHz, ensuring significant timing slack and preventing any critical path propagation delays during multi-node NoC arbitration. The signal generator was also able to achieve an $F_{\max}$ of 440MHz.

4.2 FPGA Resource Footprint Allocation

The Signal Generator isolates its 1,600 quantized sample points entirely within dedicated internal block memory bits, ensuring zero utilisation of valuable adaptive logic modules (ALMs) for data storage.



Flow Summary	
<<Filter>>	
Flow Status	Successful - Tue May 19 23:51:53 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	signal_gen
Top-level Entity Name	asp_signal
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	22 / 32,070 (< 1 %)
Total registers	54
Total pins	84 / 457 (18 %)
Total virtual pins	0
Total block memory bits	16,000 / 4,065,280 (< 1 %)
Total DSP Blocks	0 / 87 (0 %)

Figure 5. FPGA Hardware Resource Utilisation Summary for Signal Generator

The architectural co-design achieves an incredibly lightweight hardware footprint. By replacing resource-heavy software algorithms (such as deep RAM buffers, continuous window Fast Fourier Transforms, or massive shift registers) with a compact 3-register history datapath and an on-chip single-port lookup table (altsyncram megafunction), physical resource consumption on the FPGA fabric is minimised.

Flow Summary	
<<Filter>>	
Flow Status	Successful - Tue May 19 21:51:49 2026
Quartus Prime Version	18.1.0 Build 625 09/12/2018 SJ Lite Edition
Revision Name	peak_detector
Top-level Entity Name	asp_peak
Family	Cyclone V
Device	5CSEMA5F31C6
Timing Models	Final
Logic utilization (in ALMs)	80 / 32,070 (< 1 %)
Total registers	159
Total pins	83 / 457 (18 %)
Total virtual pins	0
Total block memory bits	0 / 4,065,280 (0 %)
Total DSP Blocks	0 / 87 (0 %)

Figure 6. FPGA Hardware Resource Utilisation Summary for Peak Detector

5. Conclusion

The integrated Signal Generator and Peak Detector Subsystem was successfully designed, implemented, and verified in hardware. By combining a 16 kHz harmonic-distorted ROM lookup table with a custom VHDL 3-sample sliding window, the system accurately isolates local wave crests from a noisy grid stream. A 24-bit counter tracking a 50 MHz system clock ensures precise time-stamping without arithmetic overflow. Integrated multi-block ModelSim simulations confirmed that the generator streams data deterministically, the peak detector extracts exactly 5 peak events over a 100 ms window and the output logic complies with single-cycle NoC protocol handshakes. The final architecture delivers low-latency, real-time tracking with a minimal FPGA footprint, making it fully ready for top-level HMPSoC grid-monitoring integration.